

CS 201: Data Structures & Algorithm Analysis

Lecture 1: Abstract Data Types and Array Data Structures

An Abstract Data Type (ADT) specifies a set of data items and operations without dictating the underlying implementation. Arrays are linear data structures storing elements in contiguous memory locations.

Arrays offer $O(1)$ constant time complexity for random index access. However, inserting or deleting elements from an arbitrary position requires shifting elements, yielding $O(n)$ linear time complexity.

Dynamic arrays, such as Python lists, automatically rescale their internal buffer when capacity is exceeded, providing amortized $O(1)$ append operations.

Lecture 2: Hash Tables, Collisions, and Complexity

A Hash Table (or Hash Map) is a dictionary data structure that maps keys to values using a cryptographic or integer hash function. A hash function transforms a key into an array index.

In optimal conditions, Hash Tables provide average-case $O(1)$ constant time complexity for search, insertion, and deletion operations. When two distinct keys produce the same hash index, a collision occurs.

Collision resolution techniques include Separate Chaining (using linked lists at each bucket) and Open Addressing (probing techniques like linear probing, quadratic probing, or double hashing).

Lecture 3: Binary Search Trees and Sorting Algorithms

A Binary Search Tree (BST) is a hierarchical node-based tree structure where the left child's key is strictly less than its parent's key, and the right child's key is greater than or equal.

For a balanced BST (such as an AVL tree or Red-Black tree), search, insertion, and deletion exhibit $O(\log n)$ logarithmic time complexity. In an unbalanced degenerate tree, performance degrades to $O(n)$.

Sorting algorithms reorder datasets. QuickSort and MergeSort operate via divide-and-conquer with average time complexity of $O(n \log n)$. MergeSort is stable and guarantees $O(n \log n)$ worst-case performance.